TRABAJO PRACTICO N.° 10

Integracion funcional del modulo desarrollado

Practicas Profesionalizantes II — Taller

1. Datos generales

Nombre del proyecto	OficioRed
Numero de grupo	Grupo 13
Integrantes	Leistmeister Juan / Gonzalez Lisandro / Rivero Mirian
Feature prioritaria	Buscador principal de profesionales por categoria de oficio
Tarea tecnica (TP9)	Validacion, correccion y refactorizacion de buscar.php
Tarea de integracion	Verificar que buscar.php funciona correctamente dentro del flujo real del sistema: formulario de index.php → buscar.php → base de datos → respuesta al usuario
Fecha	Junio 2026

2. Punto de partida

Version que se integra	Version refactorizada y validada al finalizar el TP9. buscar.php con partial card-profesional.php, comentarios de seccion y encabezado descriptivo. Todos los criterios de aceptacion cumplidos, cinco casos de prueba satisfactorios, sin regresiones.
Comportamiento ya probado	(1) Entrada valida "electricista" → devuelve card del profesional activo. (2) Entrada de 1 caracter "a" → mensaje de error sin consulta a BD. (3) Entrada valida sin resultados "plomero" → estado vacio con mensaje. (4) Entrada de exactamente 2 caracteres → consulta ejecutada. (5) Campo password no visible en ningun output HTML.

Pregunta central del TP10	El buscador funcionaba de forma aislada en el TP9. En este TP10 se verifica que tambien funciona dentro del flujo real del sistema: recibiendo el parametro desde el formulario de index.php, consultando la BD real y devolviendo la respuesta al usuario en la pantalla.

3. Flujo de integracion

Circuito completo extremo a extremo:

```
[1] USUARIO
Ingresa al sitio en index.php (localhost/OFICIORED/)
Escribe "electricista" en el campo de busqueda
Hace click en el boton Buscar
|
| Formulario GET
v
[2] index.php
<form action="buscar.php" method="GET" class="form-busqueda">
<input type="text" name="q" value="" ...>
<button type="submit">Buscar</button>
</form>
Envia: GET buscar.php?q=electricista
|
| HTTP GET con parametro q
v
[3] buscar.php (logica PHP)
$q = trim($_GET['q']) // "electricista"
strlen($q) >= 2 // validacion OK
$pdo->prepare(SELECT ... WHERE oficio LIKE :termino AND estado = 'activo')
$stmt->execute([':termino' => '%electricista%'])
|
| Consulta PDO preparada
v
[4] BASE DE DATOS: tabla profesionales (MySQL - officiered)
Busca registros con oficio LIKE '%electricista%' AND estado = 'activo'
Retorna: id_profesional, nombre, apellido, oficio,
zona_cobertura, telefono, email
|
| Array de resultados
v
[5] buscar.php (vista HTML)
```

```
require_once 'includes/header.php'
foreach ($resultados as $p):
include 'includes/card-profesional.php'
endforeach
|
| HTML renderizado
v
[6] USUARIO
Ve la pagina de resultados con URL: buscar.php?q=electricista
Card de "juan gonzalez" visible con oficio, zona, tel y email
```

El flujo cierra 6 elementos: usuario → formulario index.php → logica buscar.php → consulta SQL tabla profesionales → renderizado HTML buscar.php → respuesta visual al usuario. Cada flecha representa una comunicacion real entre componentes verificable con evidencia.

4. Componentes involucrados

Componente	Funcion en el flujo	Estado
index.php	Pagina de inicio. Contiene el formulario de busqueda con action="buscar.php" y method="GET". El usuario escribe el termino y hace click en Buscar.	Ya existia. Sin ajustes para esta integracion.
buscar.php	Recibe el parametro GET "q", lo valida, ejecuta la consulta PDO y renderiza los resultados como cards HTML o el estado vacio.	Ya existia. Version validada del TP9.
includes/db.php	Establece la conexion PDO a la base de datos MySQL y la deja disponible como variable $pdo para buscar.php.	Ya existia. Sin ajustes.
includes/funciones.php	Provee la funcion sanitizar() usada en todos los outputs HTML y redirigir() para el caso de parametro vacio.	Ya existia. Sin ajustes.
includes/header.php	Genera el HTML desde el DOCTYPE hasta la apertura del main. Carga CSS, fuentes y Lucide Icons.	Ya existia. Sin ajustes.
includes/footer.php	Cierra el HTML, carga main.js e inicializa Lucide Icons.	Ya existia. Sin ajustes.

includes/card-profesional.php	Partial reutilizable que renderiza una card con los datos de un profesional. Extraido en la refactorizacion del TP9.	Creado en TP9. Sin ajustes adicionales.
BD: tabla profesionales	Almacena los registros de profesionales. La consulta filtra por oficio LIKE :termino AND estado = "activo".	Ya existia. Sin modificaciones al esquema.

5. Contrato entre componentes

index.php → buscar.php	Mecanismo: formulario HTML con method="GET" y action="buscar.php". Parametro enviado: q (string, texto libre). Obligatorio: si. Minimo requerido: 2 caracteres para ejecutar la consulta. Si q esta vacio: buscar.php redirige a index.php inmediatamente. Ejemplo de URL generada: localhost/OFICIORED/buscar.php?q=electricista
buscar.php → tabla profesionales	Mecanismo: consulta PDO preparada. Query: SELECT id_profesional, nombre, apellido, oficio, zona_cobertura, telefono, email FROM profesionales WHERE oficio LIKE :termino AND estado = "activo" ORDER BY nombre ASC. Parametro: :termino = "%" + q + "%". Campos excluidos: password, foto_perfil, biografia, fecha_registro. Respuesta: array asociativo con los registros encontrados, o array vacio.
tabla profesionales → buscar.php	Respuesta: array de filas con campos id_profesional, nombre, apellido, oficio, zona_cobertura, telefono, email. Si hay resultados: se renderiza la grilla de cards. Si el array esta vacio: se muestra el estado vacio con mensaje orientador.
buscar.php → usuario (HTML)	Respuesta: pagina HTML completa con header, barra de busqueda compacta, seccion de resultados (cards o estado vacio) y footer. Los datos dinamicos pasan por sanitizar() antes de ser impresos. Lucide Icons se inicializa con lucide.createIcons() en el footer.

6. Dependencias

Dependencia	Tipo	Funcion
includes/db.php	Archivo PHP	Conexion PDO. Debe existir y tener credenciales correctas de XAMPP.
includes/funciones.php	Archivo PHP	Funciones sanitizar() y redirigir(). Incluido antes del procesamiento.
includes/header.php	Archivo PHP	HTML base, meta tags, carga de CSS y Lucide Icons CDN.
includes/footer.php	Archivo PHP	Cierre HTML, carga de main.js, lucide.createIcons().
includes/card-profesional.php	Archivo PHP (partial)	Template de la card del profesional. Requiere la variable $p definida.
css/estilos.css	Hoja de estilos	Estilos de todas las secciones: barra compacta, grid, cards, estado vacio.
js/main.js	JavaScript	Comportamientos globales: menu hamburguesa, mensajes flash, validacion del formulario de busqueda.
BD: oficiored.profesionales	Tabla MySQL	Fuente de datos. Requiere al menos un registro con estado = "activo".
Lucide Icons CDN	Libreria externa	https://unpkg.com/lucide@latest — iconos SVG. Requiere conexion a internet.
Google Fonts CDN	Fuente externa	Inter — tipografia del sitio. Requiere conexion a internet.
XAMPP (Apache + MySQL)	Entorno	Servidor local. Apache en puerto 80, MySQL en puerto 3306.

7. Cambios realizados para integrar

No fue necesario realizar cambios funcionales para lograr la integracion. Los componentes ya estaban correctamente conectados desde la version del TP9. El formulario de index.php ya apuntaba a buscar.php con method GET, buscar.php ya incluia db.php y funciones.php, y la consulta SQL ya usaba los nombres de campo correctos (zona_cobertura, estado). La integracion se verifico ejecutando el flujo completo sin modificaciones.

8. Uso de IA durante la integracion

No fue necesario utilizar IA durante la integracion. Los componentes ya estaban correctamente especificados en el Master Spec del proyecto y validados en el TP9. La verificacion del flujo se realizo directamente ejecutando el sistema en XAMPP y observando el comportamiento en el navegador.

9. Control de versiones

Mecanismo de respaldo	Antes de iniciar las pruebas de integracion se realizo una copia de seguridad de la carpeta del proyecto: C:/xampp/htdocs/OFICIORED_backup_TP10/ Esta copia conserva la version validada del TP9 como punto de retorno seguro.
Git	El repositorio en GitHub tiene el proyecto versionado con la version del TP9. Al no requerirse modificaciones funcionales durante la integracion, no se generaron commits nuevos en este TP10. La rama activa es develop.

10. Plan minimo de pruebas de integracion

Las pruebas de integracion verifican el flujo entre componentes, no cada pieza por separado. Cada caso recorre al menos tres componentes del sistema.

Caso	Tipo	Flujo recorrido	Entrada	Resultado esperado
FI-1	Valido con resultados	index.php (formulario) → buscar.php (logica) → profesionales (BD) → buscar.php (HTML) → usuario	Usuario escribe "electricista" en el formulario de la home y hace click en Buscar	URL cambia a buscar.php?q=electricista. Card de juan gonzalez visible con oficio, zona, tel y email. Sin password expuesto.
FI-2	Invalido por entrada corta	index.php (formulario) → buscar.php (validacion) → usuario	Usuario escribe "a" en el formulario de la home y hace click en Buscar	URL cambia a buscar.php?q=a. Mensaje "Escribi al menos 2 caracteres para buscar." Sin consulta a la BD.
FI-3	Valido sin resultados	index.php (formulario) → buscar.php (logica) → profesionales (BD, 0 filas) → buscar.php (HTML) → usuario	Usuario escribe "plomero" en el formulario de la home y hace click en Buscar	URL cambia a buscar.php?q=plomero. Estado vacio con mensaje orientador y boton Volver al inicio.
FI-4	Navegacion por categoria	index.php (categoria click) → buscar.php (logica) → profesionales (BD) → buscar.php (HTML) → usuario	Usuario hace click en la categoria "Electricista" desde la grilla de la home	URL cambia a buscar.php?q=Electricista. Mismo resultado que FI-1.

11. Ejecucion de las pruebas

Caso	Resultado esperado	Resultado obtenido	Estado	Evidencia
FI-1 — electricista	Card con datos del profesional activo. URL con q=electricista.	URL: buscar.php?q=electricista. Card de juan gonzalez visible con todos los campos. Sin password.	Cumple	Figura 3 y 4
FI-2 — a (1 caracter)	Mensaje de entrada invalida. Sin consulta a BD.	"Escribi al menos 2 caracteres para buscar." Visible en pantalla.	Cumple	Figura 5 (TP9)
FI-3 — plomero (sin resultados)	Estado vacio con mensaje y boton.	"No encontramos profesionales para plomero." Boton Volver al inicio.	Cumple	Figura 6 (TP9)
FI-4 — click categoria	Mismos resultados que FI-1 pero via link de categoria.	URL: buscar.php?q=Electricista. Card visible correctamente.	Cumple	Figura 4

12. Problemas de integracion detectados

No se detectaron problemas de integracion. El flujo completo funciono correctamente desde el primer intento. Los contratos entre componentes estaban correctamente definidos desde la especificacion tecnica del proyecto (Master Spec SDD).

Verificaciones realizadas	- El formulario de index.php envia el parametro "q" correctamente via GET. - buscar.php recibe el parametro y ejecuta la validacion antes de la consulta. - La consulta PDO usa zona_cobertura (nombre correcto en la tabla). - El filtro AND estado = "activo" excluye registros inactivos. - La card se renderiza desde el partial card-profesional.php sin errores. - Todos los outputs HTML pasan por sanitizar(). - El campo password no aparece en ningun output visible.

13. Correcciones y nueva validacion

No fue necesario realizar correcciones. El flujo integrado funciono correctamente desde el inicio. Se ejecutaron los cuatro casos de prueba de integracion y todos resultaron satisfactorios. No se detectaron regresiones.

14. Evidencias del flujo real

Las siguientes capturas demuestran el flujo de integracion de extremo a extremo, recorriendo todos los componentes del sistema:

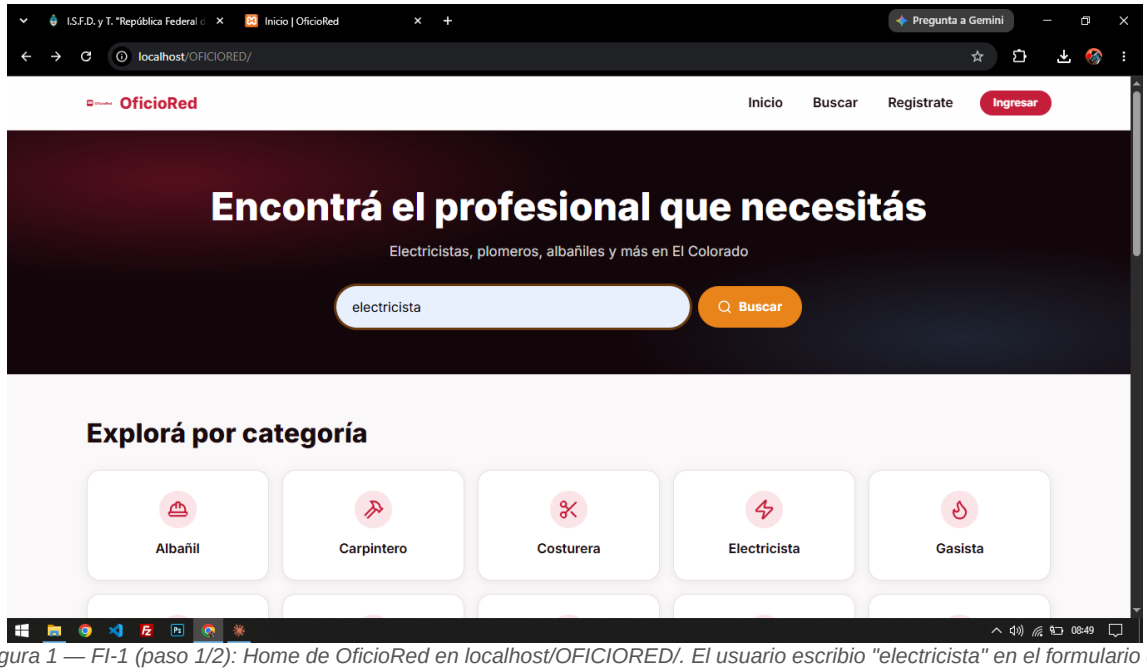


Figura 1 — FI-1 (paso 1/2): Home de OficioRed en localhost/OFICIORED/. El usuario escribio "electricista" en el formulario de busqueda de index.php. El formulario apunta a buscar.php con method GET.

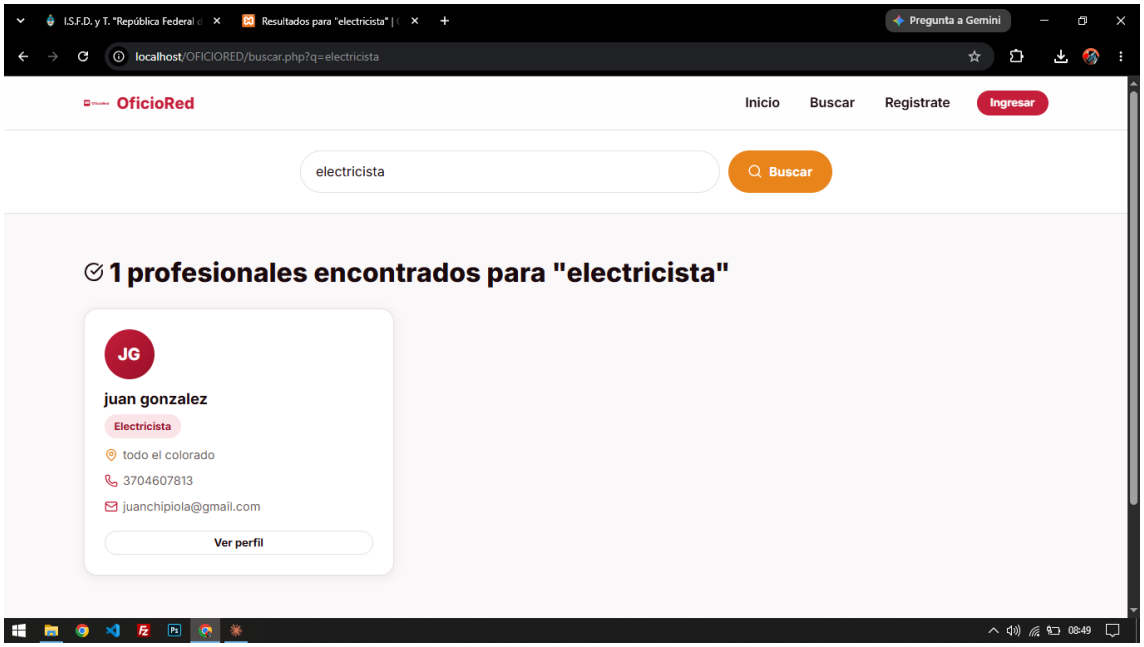


Figura 2 — FI-1 (paso 2/2): Resultado en localhost/OFICIORED/buscar.php?q=electricista. La URL muestra el parametro GET recibido desde el formulario. La card de juan gonzalez se renderiza correctamente con oficio, zona, telefono y email. Flujo index.php → buscar.php → BD → HTML completado.

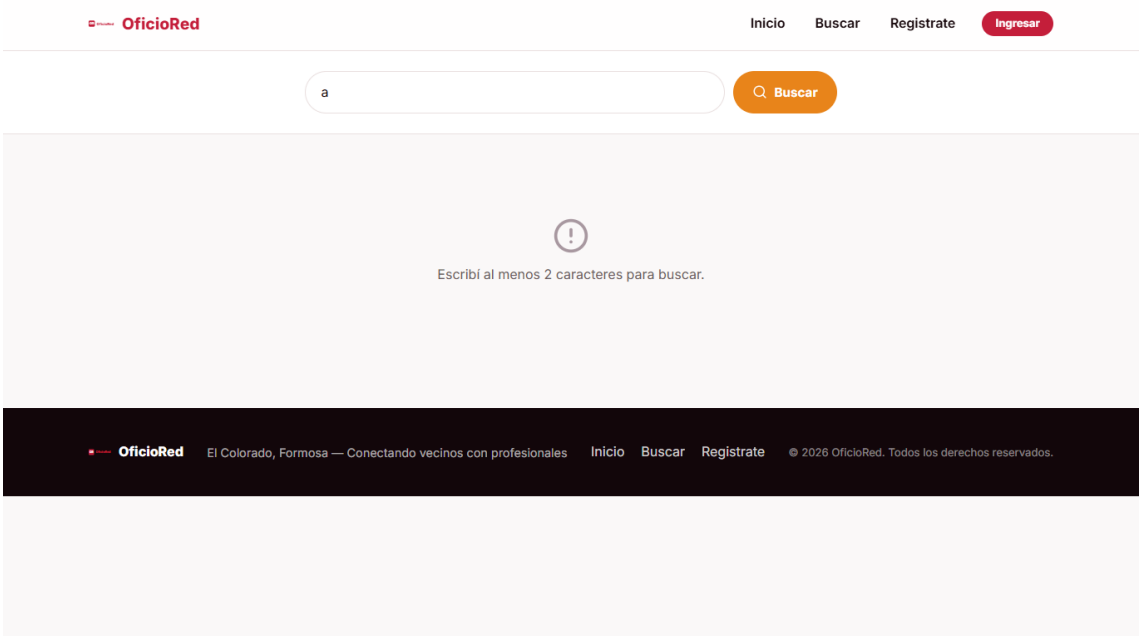


Figura 3 — FI-2: Caso invalido. Busqueda "a" (1 caracter). El sistema muestra "Escribi al menos 2 caracteres para buscar." sin ejecutar consulta a la base de datos.

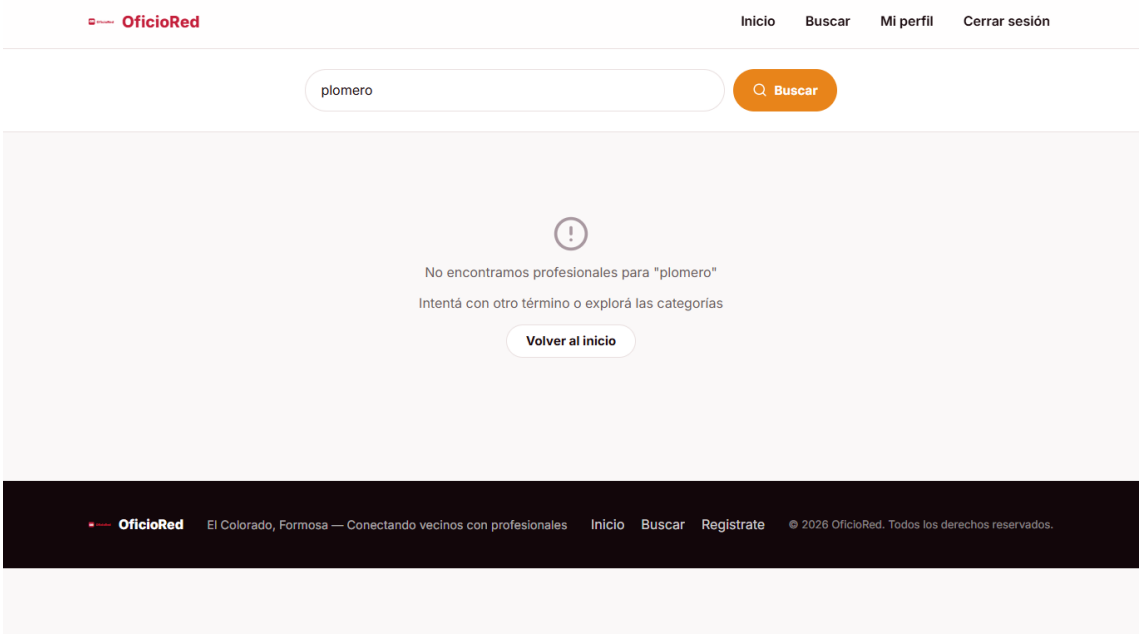


Figura 4 — FI-3: Caso valido sin resultados. Busqueda "plomero". El sistema muestra el estado vacio con mensaje orientador y boton Volver al inicio. El flujo index.php → buscar.php → BD (0 filas) → HTML funciona correctamente.

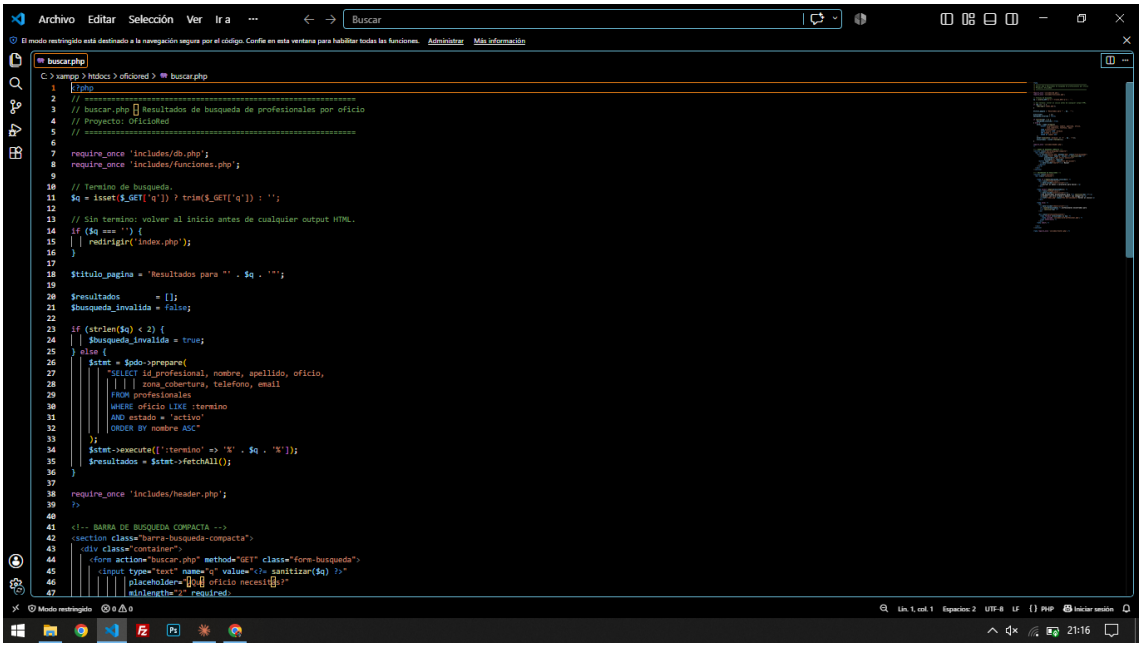


Figura 5 — Codigo fuente de buscar.php en VS Code. Se observa la inclusion de includes/db.php y includes/funciones.php, la consulta PDO con zona_cobertura y filtro AND estado = "activo", confirmando la coherencia entre el codigo y el flujo integrado.

15. Determinacion final

A. Integrada y funcional.

Determinacion	A — Integrada y funcional.
Justificacion	Los cuatro flujos de prueba de integracion fueron satisfactorios. El circuito completo index.php → buscar.php → tabla profesionales → renderizado HTML → usuario funciona correctamente en el entorno XAMPP local. Los contratos entre componentes se respetan: el parametro q llega correctamente desde el formulario, la consulta SQL usa los nombres de campo correctos, los resultados se renderizan sin exponer datos sensibles, y el estado vacio y el caso invalido funcionan sin errores. No se detectaron problemas de integracion ni regresiones.

16. Pendientes tecnicos

Pendiente	Descripcion	Impacto	Iteracion
strlen() y UTF-8	strlen() mide bytes, no caracteres visuales. Terminos con caracteres acentuados o n con tilde podrian comportarse de forma inesperada en la validacion de 2 caracteres.	No bloqueante. El caso comun de busqueda funciona correctamente.	v2 — Evaluar mb_strlen()
sanitizar() en $titulo_pagina	La variable $titulo_pagina incluye $q sin aplicar sanitizar() al momento de la asignacion. El valor se sanitiza al mostrarse en el header, pero aplicarlo en la asignacion mejoraria la claridad.	No bloqueante. Sin riesgo funcional en el estado actual.	Mejora de claridad en iteracion siguiente
Paginacion de resultados	Si la cantidad de profesionales registrados crece, la busqueda devuelve todos los resultados sin limite. Convendria incorporar LIMIT y OFFSET en versiones futuras.	No bloqueante en la etapa actual con datos de prueba.	v2 del proyecto
Busqueda por nombre/apellido	El buscador filtra unicamente por el campo oficio. Un usuario podria querer buscar por el nombre de un profesional conocido.	No bloqueante. El caso de uso principal (buscar por oficio) funciona.	v2 del proyecto

Checklist antes de entregar

[+] Trabajamos sobre la misma tarea tecnica validada en el TP9 (buscar.php).

[+] Identificamos el flujo real de integracion (6 componentes, de extremo a extremo).

[+] Definimos los 8 componentes involucrados con su funcion y estado.

[+] Documentamos el contrato entre los 4 pares de componentes del flujo.

[+] Identificamos las 11 dependencias necesarias para el funcionamiento.

[+] Conservamos una version segura antes de integrar (backup + Git).

[+] No fue necesario realizar cambios funcionales para lograr la integracion.

[+] No fue necesario utilizar IA durante la integracion.

[+] Definimos 4 flujos de prueba (valido con resultados, invalido, sin resultados, categoria).

[+] Ejecutamos los cuatro flujos de prueba con resultado satisfactorio.

[+] No se detectaron problemas de integracion.

[+] No fue necesario corregir nada — no se generaron regresiones.

[+] Adjuntamos 5 evidencias reales del flujo integrado en XAMPP.

[+] Determinacion A: integrada y funcional, con justificacion.

[+] Registramos 4 pendientes tecnicos no bloqueantes para iteraciones futuras.